IT Case Manager

Project Handover Document

Project: Internal IT Case Management System

Repository: github.com/aivarya/Case_Management

Version: 1.0

Date: March 2026

Prepared by: Sreyas Sambhu

Confidential — Internal Use Only

Table of Contents

1. 1. Project Overview
2. 2. Technology Stack
   - Frontend, Backend, Infrastructure
3. 3. Project Structure
4. 4. Database Schema
   - User, Ticket, Comment, ActivityLog
5. 5. Backend API Reference
   - Authentication, Tickets, Users
6. 6. Frontend Pages & Components
   - Login, Dashboard, Kanban, Calendar, List View, Admin Panel
   - Navbar, TicketCard, TicketModal, CommentThread
7. 7. Authentication & Authorization
8. 8. Key Features
9. 9. Business Logic
10. 10. Docker & Deployment
11. 11. Environment Variables
12. 12. Running the App
13. 13. Default Login Credentials

1 Project Overview

The IT Case Manager is a full-stack web application built for internal IT departments to systematically manage support cases and tasks. It is designed to replace informal tracking methods (email, spreadsheets) with a structured, role-based ticketing system similar to Jira or Monday.com.

What It Does

- Create, assign, and track IT support cases (tickets) with priority levels and due dates
- Visualise the team's workload across multiple views: Kanban, Calendar, and List
- Enforce resolution documentation before a ticket can be closed
- Manage team members and control access through role-based permissions
- Export ticket reports to CSV and Excel for management reporting
- Track all activity, status changes, and comments per ticket

2 Technology Stack

Frontend

Package	Version	Purpose
React	18.3.1	UI framework
Vite	5.3.1	Build tool and development server
React Router DOM	6.24.0	Client-side page routing
@hello-pangea/dnd	16.6.0	Drag-and-drop for the Kanban board
react-big-calendar	1.13.1	Calendar view
date-fns	3.6.0	Date formatting utilities

Backend

Package	Version	Purpose
Node.js	18+	JavaScript runtime
Express	4.19.2	Web server framework
Prisma	5.14.0	ORM / database client
PostgreSQL	16	Relational database
bcryptjs	2.4.3	Password hashing
express-session	1.18.0	Session-based authentication
CORS	2.8.5	Cross-origin request handling

Infrastructure

Tool	Purpose
Docker	Containerises all services
Docker Compose	Orchestrates multi-container setup
Nginx	Serves the frontend and proxies API requests to backend
GitHub	Source code repository (github.com/aivarya/Case_Management)

3 Project Structure

```
case-management-app/
├── docker-compose.yml          # Orchestrates all 3 containers
├── .gitignore
├── .env.example
├── README.md
├── DOCUMENTATION.md
│
├── backend/
│   ├── server.js               # Express app entry point
│   ├── seed.js                 # Creates default admin user
│   ├── package.json
│   ├── Dockerfile
│   ├── .env                    # NOT in git — must be created manually
│   ├── middleware/
│   │   └── auth.js             # requireAuth / requireAdmin middleware
│   ├── routes/
│   │   ├── auth.js             # Login, logout, me, change password
│   │   ├── tickets.js          # Ticket CRUD operations
│   │   ├── users.js            # User management (admin)
│   │   └── comments.js         # Ticket comments
│   └── prisma/
│       ├── schema.prisma       # Database models definition
│       └── migrations/         # SQL migration history
│
└── frontend/
    ├── package.json
    ├── vite.config.js
    ├── Dockerfile
    ├── nginx.conf              # Nginx config and API proxy rules
    └── src/
        ├── App.jsx             # Router + AuthContext
        ├── api.js              # Centralised API client
        ├── pages/
        │   ├── Login.jsx
        │   ├── Dashboard.jsx
        │   ├── KanbanBoard.jsx
        │   ├── CalendarView.jsx
        │   ├── ListView.jsx
        │   ├── AdminPanel.jsx
        │   └── CreateTicketForm.jsx
        ├── components/
        │   ├── Navbar.jsx
        │   ├── TicketCard.jsx
        │   ├── TicketModal.jsx
        │   └── CommentThread.jsx
        └── styles/
            └── global.css      # Dark theme global styles
```

4 Database Schema

User

Field	Type	Notes
id	Int	Auto-increment primary key
email	String	Unique — used for login
name	String	Display name
passwordHash	String	bcrypt hashed password
role	Enum	ADMIN or AGENT
disabled	Boolean	Default false — blocks login when true
createdAt	DateTime	Auto-set on creation

Ticket

Field	Type	Notes
id	Int	Auto-increment — used as Ticket ID
title	String	Required
description	String	Optional
requestor	String	Who raised the request
priority	Enum	LOW, MEDIUM, HIGH, CRITICAL
status	Enum	TODO, IN_PROGRESS, IN_REVIEW, DONE
resolution	String	Required when status = DONE
dueDate	DateTime	Optional
createdAt	DateTime	System-generated on creation
updatedAt	DateTime	Auto-updated on every save
createdById	Int	FK → User
assignedToId	Int	FK → User (nullable — can be unassigned)

Comment

Field	Type	Notes
id	Int	Auto-increment primary key
body	String	Comment text
createdAt	DateTime	Auto-set
ticketId	Int	FK → Ticket (cascade delete)
authorId	Int	FK → User

ActivityLog

Field	Type	Notes
id	Int	Auto-increment primary key
action	String	e.g. "Status changed from TODO to DONE"
createdAt	DateTime	Auto-set
ticketId	Int	FK → Ticket (cascade delete)
performedById	Int	FK → User

5 Backend API Reference

Authentication — /api/auth

Method	Endpoint	Access	Description
POST	/login	Public	Login with email + password
POST	/logout	Auth	Destroy session and log out
GET	/me	Auth	Get current authenticated user
PATCH	/password	Auth	Change own password (requires current password)

Tickets — /api/tickets

Method	Endpoint	Access	Description
GET	/	Auth	List tickets — Admins see all, Agents see assigned only
POST	/	Auth	Create a new ticket
GET	/:id	Auth	Get ticket detail with comments and activity log
PATCH	/:id	Auth	Update ticket fields
DELETE	/:id	Admin	Delete a ticket
POST	/:id/comments	Auth	Add a comment to a ticket

Users — /api/users

Method	Endpoint	Access	Description
GET	/	Auth	List users — Admins: full details; Agents: name/email only
POST	/	Admin	Create a new user
PATCH	/:id/disable	Admin	Toggle user disabled status
DELETE	/:id	Admin	Delete user with cascading data cleanup

6 Frontend Pages & Components

Pages

Login

- Email and password form with error handling
- Redirects to Dashboard on successful login

Dashboard

- Summary cards: Open Tickets, Assigned to Me, Resolved, Critical Open
- Recently Updated Tickets and Assigned to Me sections
- New Ticket quick-create button

Kanban Board

- Four columns: To Do, In Progress, In Review, Done
- Drag-and-drop tickets between columns
- Ticket count badges per column
- Click any ticket to open the full detail modal

Calendar View

- Monthly calendar showing tickets on their due dates
- Colour-coded by priority: Green (Low), Blue (Medium), Orange (High), Red (Critical)
- Supports Month, Week, and Agenda views

List View

- Sortable table: ID, Title, Requestor, Priority, Status, Assigned To, Created, Due Date, Resolution
- Search filter across title, requestor, and assignee
- Filter by Status and Priority
- Export filtered list to CSV or Excel
- Overdue tickets highlighted in red

Admin Panel

- Team members table with Name, Email, Role, Status, Joined date
- Enable/Disable and Delete actions per user
- Disabled users shown dimmed with a Disabled badge (not hidden)
- Add Team Member form with name, email, temporary password, and role

Components

Navbar

- Navigation links: Dashboard, Kanban, Calendar, List, Admin (Admin only)
- Displays logged-in user name and role badge
- Change Password button (available to all users)

- Logout button

TicketModal

- Full ticket editor: Title, Requestor, Description, Priority, Status, Assigned To, Due Date
- Resolution field appears and is required when Status = DONE
- Ticket ID and Creation Date shown as read-only system fields
- Disabled users excluded from the Assigned To dropdown
- Automatically closes after successful save or delete

CommentThread

- Displays all comments with author and timestamp
- Activity log showing all status change history

7 Authentication & Authorization

How It Works

- Session-based authentication using `express-session`
- Passwords hashed with `bcryptjs` (12 salt rounds)
- Sessions persist for 7 days
- All API requests automatically send session cookies
- Unauthenticated requests are redirected to the login page

User Roles

Role	Capabilities
ADMIN	See all tickets, manage users, access Admin Panel, delete tickets and users
AGENT	See only assigned tickets, update ticket status, add comments, change own password

Security Rules

- Disabled users receive a 403 error on login — cannot access the system
- Agents cannot access the `/admin` route — redirected to Dashboard
- Agents cannot call admin-only API endpoints — receive 403 response
- Users cannot disable or delete their own account

8 Key Features

Ticket Management

- Create tickets with Title, Requestor, Description, Priority, Assigned To, Due Date
- System auto-generates Ticket ID and Creation Date
- Resolution field mandatory before a ticket can be marked Done
- Status workflow: **To Do** → **In Progress** → **In Review** → **Done**

Multiple Views

- **Dashboard** — summary stats and recent activity at a glance
- **Kanban Board** — drag-and-drop workflow management
- **Calendar View** — timeline view for deadline management
- **List View** — detailed table with search, filters, sorting, and export

Export

- Export current filtered ticket list to CSV
- Export current filtered ticket list to Excel (.xls)
- Files automatically named with today's date

Team Management (Admin)

- Create team members with a temporary password
- Assign roles: Admin or Agent
- Disable users — blocks login and hides from assignment, but preserves all data
- Re-enable disabled users at any time
- Delete users — all related tickets are safely reassigned

Collaboration

- Post comments on any ticket
- Automatic activity log records all status changes with timestamp and actor

9 Business Logic

Ticket Visibility by Role

```
ADMIN → sees ALL tickets in the system AGENT → sees ONLY tickets assigned to them
```

Resolution Enforcement

- The Resolution field only appears when Status is set to DONE
- Resolution text is mandatory — save is blocked on both client and server if empty
- Ensures every closed ticket has a documented resolution before it is closed

User Deletion Cascade

When an admin deletes a user, the following cleanup steps run in order:

1. All tickets **assigned to** the user are unassigned
2. All tickets **created by** the user are reassigned to the deleting admin
3. All **comments** by the user are deleted
4. All **activity log** entries by the user are deleted
5. The user record is deleted

Assignment Dropdown Rules

- Only active (non-disabled) users appear in the Assigned To dropdown
- Applies to both Create Ticket and Edit Ticket forms
- Tickets can be left unassigned

Overdue Detection

- Calculated client-side: `dueDate < today AND status !== DONE`
- Overdue tickets highlighted in red in List View
- Warning icon shown on Kanban ticket cards

10 Docker & Deployment

System Architecture

```
Browser └── http://localhost (port 80) └── Nginx (frontend container) ├── / → serves React SPA (static files) └── /api/* → proxied to backend:3001 └── Express + Prisma (backend container) └── PostgreSQL 16 (db container)
```

Docker Compose Services

Service	Image	Port	Description
db	postgres:16	Internal only	PostgreSQL database with persistent volume
backend	Custom build	Internal (3001)	Node.js + Express API server
frontend	Custom build	80:80	Nginx serving the React app

Backend Dockerfile Startup Sequence

1. Base image: `node:18-slim` (Debian — required for Prisma SSL compatibility)
2. Installs `openssl` and `postgresql-client`
3. Copies code and runs `npm install`
4. Generates Prisma client
5. On container start: waits for PostgreSQL to be ready
6. Runs `prisma migrate deploy` to apply schema migrations
7. Runs `node seed.js` to create default admin if not exists
8. Starts `node server.js`

Commands

Command	When to Use
`docker-compose up --build`	First run or after any code changes
`docker-compose up`	Normal start (no code changes)
`docker-compose down`	Stop all containers
`docker-compose logs backend`	View backend logs for debugging

11 Environment Variables

Backend — backend/.env

> ⚠️ This file is excluded from the git repository. It must be created manually on every machine where the app is deployed.

```
DATABASE_URL="postgresql://postgres:secret@localhost:5432/casemgmt"
SESSION_SECRET="change-this-to-a-random-string"
PORT=3001
```

Variable	Description
DATABASE_URL	PostgreSQL connection string. Use `db` as host when running in Docker
SESSION_SECRET	Secret key for signing session cookies — change this in production
PORT	Port the backend listens on (default: 3001)

Frontend

```
VITE_API_URL="/api"    # Optional — defaults to /api if not set
```

12 Running the App

With Docker (Recommended)

> Requirements: Git and Docker Desktop installed

#	Command	Purpose
1	`git clone https://github.com/aivarya/Case_Management.git`	Download the code
2	`cd Case_Management`	Enter project folder
3	Ensure Docker Desktop is running	Required before next step
4	`docker-compose up --build`	Build and start all containers
5	Wait ~3–5 minutes on first run	Downloads images, builds, migrates DB
6	Open `http://localhost` in browser	Access the application

Without Docker (Development Mode)

> Requirements: Node.js 18+, PostgreSQL running locally

```
# Terminal 1 — Backend
cd backend
npm install
npx prisma migrate deploy
node seed.js
node server.js

# Terminal 2 — Frontend
cd frontend
npm install
npm run dev
```

Open browser: `http://localhost:5173`

13 Default Login Credentials

URL

http://localhost

EMAIL

admin@company.com

PASSWORD

admin123

ROLE

ADMIN

⚠️ Change the admin password immediately after first login. Use the **Password** button in the top navigation bar.